Practice_Day2

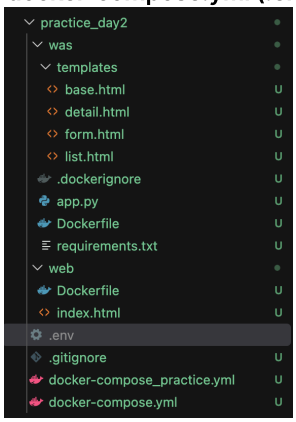
1. 실습

1.1. 지난 시간 및 오늘 오전에 수행한 Container 및 network 정리

```
● → practice_day2 git:(main) ✗ docker rm -f practice_day2-web-1 demo board-web board-was
  practice_day2-web-1
  demo
  board-web
  board-was
● → practice_day2 git:(main) ✗ docker ps -a
  CONTAINER ID   IMAGE         COMMAND                  CREATED        STATUS                     PORTS     NAMES
  8bd2c9f530f1   postgres:17   "docker-entrypoint.s…"   23 hours ago   Exited (0) 21 hours ago              board-db
  9801f8d03d39   alpine        "sleep 1000"             24 hours ago   Exited (0) 24 hours ago              wonderful_edison
  81e170d41b1a   hello-world   "/hello"                 27 hours ago   Exited (0) 27 hours ago              quirky_shamir
● → practice_day2 git:(main) ✗ docker rm -f board-db
  board-db
● → practice_day2 git:(main) ✗ docker ps -a
  CONTAINER ID   IMAGE         COMMAND        CREATED        STATUS                     PORTS     NAMES
  9801f8d03d39   alpine        "sleep 1000"   24 hours ago   Exited (0) 24 hours ago              wonderful_edison
  81e170d41b1a   hello-world   "/hello"       27 hours ago   Exited (0) 27 hours ago              quirky_shamir
```

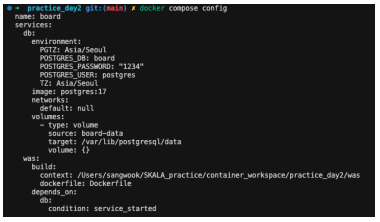
```
  81e170d41b1a   hello-world   "/hello"       27 hours ago   Exited (0) 27 hours
● → practice_day2 git:(main) ✗ docker network rm board-net
  board-net
```

1.2. docker-compose.yml (.env 활용)



전체적인 folder 구조





docker compose config를 통해 위와 같이 값이 .env를 활용해 치환됨을 알 수 있음

1.3. docker compose build를 통해 web, was 이미지 새로 빌드

```
● → practice_day2 git:(main) ✗ docker compose build
  [+] Building 2.9s (22/22) FINISHED
   => [internal] load local bake definitions
   => => reading from stdin 1.06kB
   => [was internal] load build definition from Dockerfile
   => => transferring dockerfile: 446B
   => [web internal] load build definition from Dockerfile
   => => transferring dockerfile: 131B
   => [web internal] load metadata for docker.io/library/nginx:alpine
   => [was internal] load metadata for docker.io/library/python:3.10-slim
   => [auth] library/nginx:pull token for registry-1.docker.io
   => [auth] library/python:pull token for registry-1.docker.io
   => [was internal] load .dockerignore
   => => transferring context: 215B
   => [was 1/5] FROM docker.io/library/python:3.10-slim@sha256:a78e4529630cfe8c5199cafd6e0c28ee1579a13f86274396d8b6b2d80367aa3a
   => => resolve docker.io/library/python:3.10-slim@sha256:a78e4529630cfe8c5199cafd6e0c28ee1579a13f86274396d8b6b2d80367aa3a
   => [was internal] load build context
   => => transferring context: 24.18kB
   => [web internal] load .dockerignore
   => => transferring context: 2B
   => [web internal] load build context
   => => transferring context: 5.66kB
   => CACHED [web 1/2] FROM docker.io/library/nginx:alpine@sha256:db35bfc6b2951e7f8a72db5db120288c127ffaeeb4a6d4b95a26fead017d5913
   => => resolve docker.io/library/nginx:alpine@sha256:db35bfc6b2951e7f8a72db5db120288c127ffaeeb4a6d4b95a26fead017d5913
   => CACHED [was 2/5] WORKDIR /app
   => CACHED [was 3/5] COPY requirements.txt .
   => CACHED [was 4/5] RUN pip install --no-cache-dir -r requirements.txt
   => [was 5/5] COPY . .
   => [web 2/2] COPY index.html /usr/share/nginx/html/index.html
   => [was] exporting to image
   => => exporting layers
   => => exporting manifest sha256:0d92429001a812d682c394b684459af23b14a725938859251f3ae2af58a23415
   => => exporting config sha256:ce15a43c533bc61c9a8e9dead46ae8eef38facbd025e32f1eae6fd4c7c024b7a
   => => exporting attestation manifest sha256:854501029b53d1d02a1d0b4e0957f8c2c13efbc91f0797eeaaad79d0da8c8518e
   => => exporting manifest list sha256:669b46dee89d9625a11639269936ba72b9a590c03f9317259e1e55ac6863603c
   => => naming to docker.io/library/board-was:latest
   => => unpacking to docker.io/library/board-was:latest
   => [web] exporting to image
   => => exporting layers
   => => exporting manifest sha256:90c5a24b8dd777943538b1599d8172f08c78483033ad707302c76f2352e05dcf
   => => exporting config sha256:7006d356bbe2cc5189a5bafc8c11ef71bad2611323740b7f9350d815e8a4572c
   => => exporting attestation manifest sha256:7122223397d960122155a1ec454aa23d064abe36e42863bd4d919be343d0e6ac2
   => => exporting manifest list sha256:49256afc0fa5995a24e2923be1aaf520fdc853f7dd795a4fff57aa230b9f2c04
   => => naming to docker.io/library/board-web:latest
   => => unpacking to docker.io/library/board-web:latest
   => [was] resolving provenance for metadata file
   => [web] resolving provenance for metadata file
  [+] build 2/2
   ✔ Image board-web Built
   ✔ Image board-was Built
```

1.4. docker compose up -d 로 세 서비스를 모두 실행

```
● → practice_day2 git:(main) ✗ docker compose up -d
 [+] up 5/5
  ✔ Network board_default    Created
  ✔ Volume board_board-data  Created
  ✔ Container board-db-1     Started
  ✔ Container board-was-1    Started
  ✔ Container board-web-1    Started
```

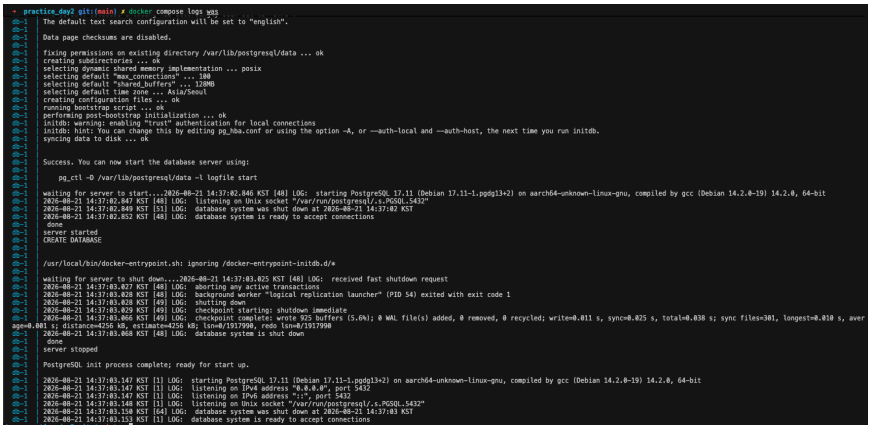
1.5. docker compose ps 로 세 서비스 모두 Up 상태인지 확인

```
● → practice_day2 git:(main) ✗ docker compose ps
NAME         IMAGE          COMMAND                  SERVICE   CREATED              STATUS              PORTS
board-db-1   postgres:17    "docker-entrypoint.s…"   db        About a minute ago   Up About a minute   5432/tcp
board-was-1  board-was      "python app.py"          was       About a minute ago   Up About a minute   0.0.0.0:5001->5000/tcp, [::]:5001->5000/tcp
board-web-1  board-web      "/docker-entrypoint.…"   web       About a minute ago   Up About a minute   0.0.0.0:8080->80/tcp, [::]:8080->80/tcp
```

1.6. docker compose logs 확인 (was, db)

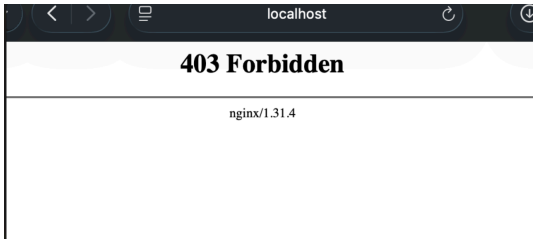
```
● → practice_day2 git:(main) ✗ docker compose logs was
was-1  | [DB 연결 대기] 1/30 - db:5432 (OperationalError)
was-1  | [DB 연결 성공] db:5432 접속 완료
was-1  |  * Serving Flask app 'app'
was-1  |  * Debug mode: off
was-1  | WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
was-1  |  * Running on all addresses (0.0.0.0)
was-1  |  * Running on http://127.0.0.1:5000
was-1  |  * Running on http://172.18.0.3:5000
was-1  | Press CTRL+C to quit
```

DB에 연결된 상태를 확인



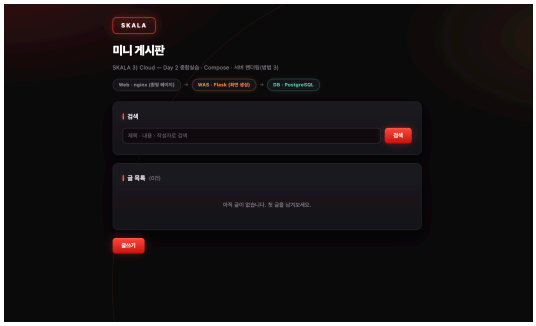
was log를 통해 확인

1.7. localhost:8080 Error 및 정상 화면

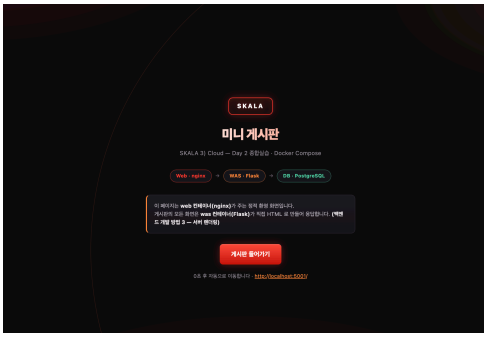


정적 파일의 권한이 `600` 으로 설정되어 **Nginx Worker (`nginx` 사용자가 파일을 읽지 못하는 문제 발생)**

자세한 부분은 2.Appendix → 2.1 Error 원인 및 해결(0.7)에서 다룰 예정

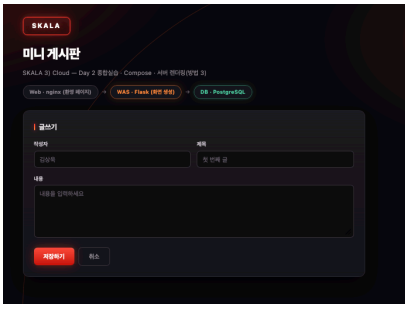


로딩 후 화면



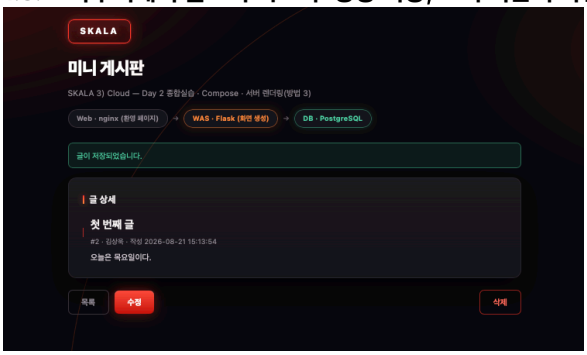
첫 로딩 화면

→ (원래는 접속하게 되면 3초가 있다가 미니 게시판을 작성, 수정, 조회할 수 있는 UI로 변경됨)

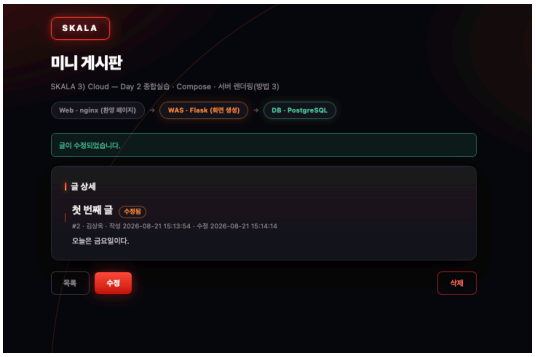


게시물을 올리기 위한 글쓰기 화면

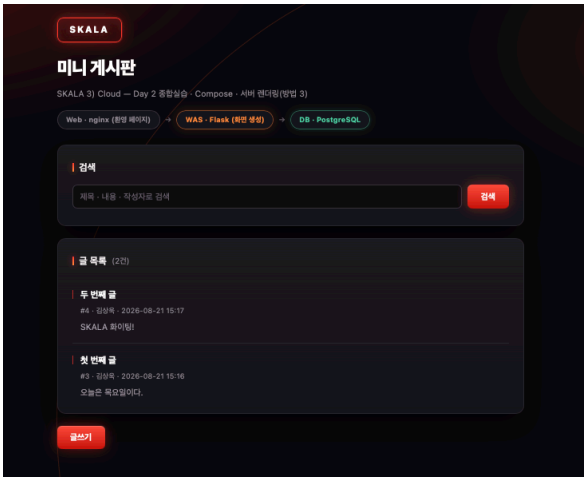
1.8. 브라우저에서 글쓰기 시도 후 정상 저장, 조회 되는지 확인



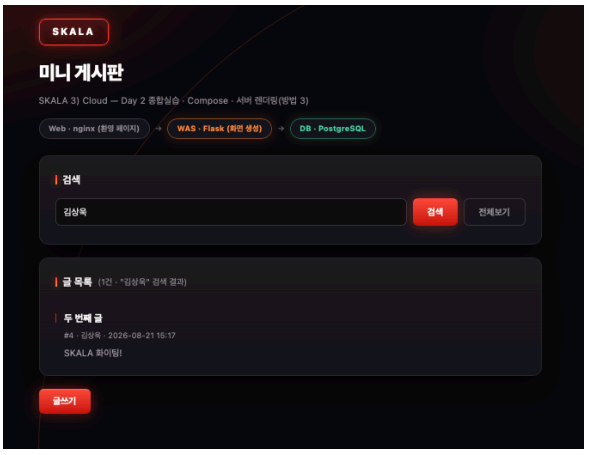
첫 번째 글이 “오늘은 목요일이다.” 라고 게시글이 나옴



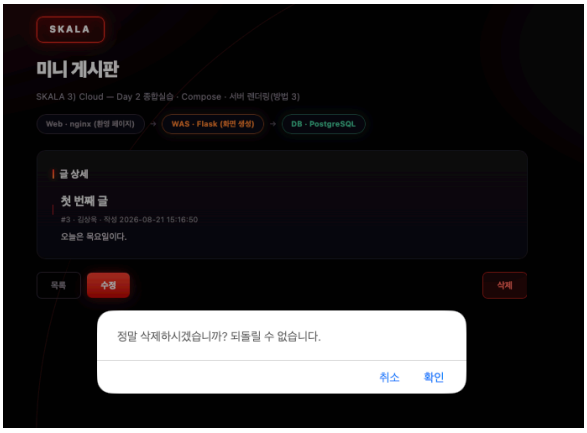
첫 번째 글에 오류가 있어 “오늘은 금요일이다.” 라고 수정



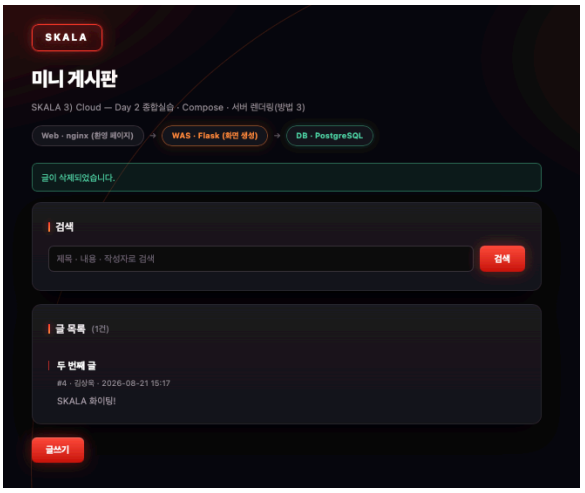
두 번째 글로 “SKALA 화이팅!” 이라는 게시글이 생성되고 목록을 통해 해당 화면을 볼 수 있음



검색 기능으로 “김상욱”을 조회하면 위와 같이 1건이 조회됨
→ (삭제 과정을 거친 후 진행했기 때문에 1건만 조회되는 것이 맞음)

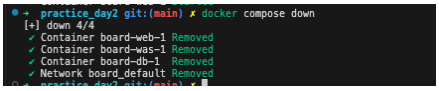


첫 번째 글 “오늘은 목요일이다”라는 게시글을 삭제 시도
→ (해당 화면은 수정 후 삭제 확인을 위해 미리 진행하고 다시 만든 상황이기 때문에 목요일로 표시됨)

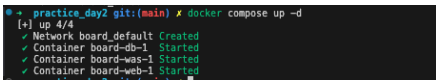


삭제 후 첫 번째 게시글이 없어짐

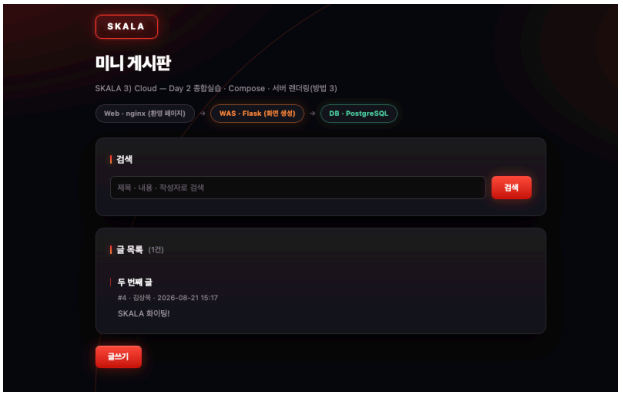
1.9. docker compose down 실행 & docker compose up -d 로 세 서비스를 모두 재실행



compose down



compose up -d



이후 localhost:5001에서도 Volume 덕분에 유지된 것을 확인
→ (Day 1은 브라우저에서 데이터를 받아 화면을 구성하고, Day 2는 WAS에서 완성된 HTML을 생성하여 브라우저에 전달한다. 이에 따라 화면 렌더링 책임이 Web에서 WAS로 이동)

자세한 내용은 2.Appendix → 2.2. Day 1과 Day 2의 렌더링 구조 변화

2. 코드 리뷰

2.1. 코드 리뷰 종합 표

검토 항목	내가 동료 코드를 리뷰한 결과	동료가 내 코드를 리뷰한 결과
`services` 하위 들여쓰기	충족 – YAML 들여쓰기 일관성 확인	충족 – 2칸 단위의 일관된 들여쓰기 확인
`build` / `image` 설정	충족 – `web`, `was`는 `build`, `db`는 `postgres:17` 사용	충족 – `./web`, `./was` 경로 및 `postgres:17` 이미지 확인
`depends_on` 설정	충족 – `web → was → db` 의존 관계 확인	충족 – `web → was → db` 구조 확인. 단, `depends_on`만으로 준비 완료는 보장되지 않음을 확인
`DB_HOST`	충족 – `DB_HOST=db`가 Compose 서비스명과 일치	충족 – `DB_HOST=db`가 `db` 서비스명과 일치하고 실제 연결 로그로 검증
DB 환경 변수	충족 – PostgreSQL 비밀번호와 DB 이름 설정 확인	충족 – `DB_NAME`, `DB_USER`, `DB_PASSWORD`까지 WAS에 전달하여 DB 접속 정보가 일관되게 관리됨
환경 변수 관리	개선 여지 – 비밀번호가 `mypassword`로 Compose 파일에 직접 작성됨	장점 – `${POSTGRES_PASSWORD}`, `${POSTGRES_DB}` 등을 사용하여 환경별 설정을 분리
`volumes`	충족 – 서비스 내부와 최상위에 모두 선언	충족 – `board-data`를 PostgreSQL 데이터 디렉터리에 연결하고 최상위에서 선언

2.2 코드 리뷰 상세

2.2.1 My → Peer 코드 리뷰

`docker-compose.yml`을 검토한 결과, 가장 큰 장점은 Health Check를 통한 서비스 상태 관리였다.

`web`, `was`, `db`에 각각 `healthcheck`를 적용하고, `depends_on`의 `condition: service_healthy`를 사용하여 의존 서비스가 실제로 준비된 이후 실행되도록 구성하였다.

특히 DB는 `pg_isready`를 사용하여 PostgreSQL의 연결 가능 여부를 확인하였다.

다만 PostgreSQL 비밀번호가 Compose 파일에 `mypassword`로 직접 작성되어 있어, 실제 환경에서는 `.env` 또는 Secret 등을 활용하여 민감 정보를 분리할 필요가 있다.

2.2.2 Peer → My 코드 리뷰

`docker-compose.yml`은 `${...}` 형태의 환경 변수를 사용하여 DB 이름, 사용자, 비밀번호, 포트 등의 설정을 Compose 파일과 분리한 점이 장점으로 확인되었다.

이를 통해 `.env`의 값만 변경하여 환경에 따라 설정을 유연하게 변경할 수 있다.

반면 `healthcheck`가 적용되어 있지 않아 DB 컨테이너가 실행된 직후 WAS가 연결을 시도할 가능성이 있다. 현재는 WAS의 DB 연결 재시도 로직을 통해 이를 보완하고 있으며, 향후 `healthcheck`와 `condition: service_healthy`를 적용하여 Compose 수준에서도 서비스 준비 상태를 관리할 수 있다.

2.1.3 코드리뷰를 통한 개선점

교차 코드리뷰를 통해 두 코드가 서로 다른 방식으로 서비스 안정성과 환경 설정 관리를 해결하고 있음을 확인하였다. 동료 코드의 `healthcheck`를 적용하여 서비스 실행 안정성을 높이고, 내 코드의 환경 변수 분리 방식을 활용하여 민감 정보와 환경별 설정을 관리한다면 두 방식의 장점을 함께 활용할 수 있다.

3. Appendix

3.1. Error 원인 및 해결(0.7)

3.1.1 문제 현상

`web` 컨테이너에서 Nginx를 통해 정적 파일에 접근할 때 HTTP 403 Forbidden 오류가 발생하였다.

3.1.2 원인 분석

컨테이너 이미지 내부의 웹 파일 권한을 확인한 결과, 파일 권한이 `-rw-------`(600)으로 설정되어 있어 소유자만 파일을 읽을 수 있는 상태였다.

`nginx:alpine` 컨테이너에서는 Nginx Master Process가 root로 실행되지만, 실제 HTTP 요청을 처리하는 Worker Process는 보안상의 이유로 `nginx` 사용자 권한으로 실행된다. 따라서 root가 소유한 600 권한의 파일을 Nginx Worker가 읽지 못해 403 오류가 발생하였다.

반면 WAS 컨테이너의 경우 Python 애플리케이션이 root 권한으로 실행되고 있어 동일한 파일 권한에서도 정상적으로 파일을 읽을 수 있었다.

3.1.3 해결 방법

웹 서버와 WAS에서 사용하는 파일 및 디렉터리에 적절한 읽기 및 접근 권한을 부여하기 위해 다음 명령어를 실행하였다.

```
chmod -R a+rX web was
```

`a+rX` 옵션을 사용하여 모든 사용자에게 파일 읽기 권한을 부여하고, 디렉터리에는 접근(`execute`) 권한을 부여하였다. 이후 변경된 파일 권한이 Docker 이미지에 반영되도록 `web` 이미지를 다시 빌드하고 컨테이너를 재기동하였다.

```
docker compose build web
docker compose up -d
```

3.1.4 해결 결과

파일 권한을 수정하고 `web` 이미지를 재빌드한 후 Nginx Worker Process에서도 정적 파일을 정상적으로 읽을 수 있게 되었으며, 기존에 발생하던 403 Forbidden 오류가 해결되었다.

이번 문제를 통해 Docker 환경에서는 단순히 파일이 컨테이너 내부에 존재하는 것뿐만 아니라, 컨테이너에서 실제 요청을 처리하는 프로세스의 실행 사용자와 파일 시스템 권한을 함께 고려해야 한다는 점을 확인하였다.

3.2. Day 1과 Day 2의 렌더링 구조 변화

Day 1과 Day 2의 가장 큰 차이는 화면을 누가 생성하는가에 있다. Day 1에서는 브라우저의 JavaScript가 API에서 데이터를 받아 화면을 구성하는 반면, Day 2에서는 Flask가 데이터를 조회하고 완성된 HTML을 생성하여 브라우저에 전달한다.

이러한 렌더링 주체의 변화는 단순히 화면 구현 방식만 변경한 것이 아니라, 브라우저의 접속 위치와 각 컨테이너의 역할까지 변화시켰다.

3.2.1 Day 1과 Day 2의 화면 렌더링 방식

Day 1의 방법 1에서는 Nginx가 `index.html`을 브라우저에 전달하고, 이후 JavaScript가 `5001`의 Flask API에서 JSON 데이터를 가져와 DOM을 구성하였다.

```
브라우저 (localhost:8080)
 ├ Nginx → index.html 전달
 └ JavaScript → localhost:5001/api/posts 요청
                    ↓
                JSON 응답
                    ↓
            DOM 구성 및 렌더링
```

따라서 Day 1에서는 `web`이 화면의 기본 구조와 JavaScript를 제공하고, `was`는 데이터를 제공하는 API 서버 역할을 담당하였다.

반면 Day 2의 방법 3에서는 Flask가 Jinja2 템플릿과 DB 데이터를 이용하여 목록, 상세 페이지, 입력 폼 등의 완성된 HTML을 서버에서 생성한다.

```
브라우저 (localhost:5001)
       ↓
     Flask
       ├ DB 데이터 조회
       ├ Jinja2 템플릿 렌더링
       └ 완성된 HTML 응답
              ↓
          브라우저
```

즉, Day 1에서는 Client Side Rendering, Day 2에서는 Server Side Rendering에가까운구조로 변경되었으며, 화면을 생성하는 책임이 `web`에서 `was`로 이동하였다.

3.2.2 브라우저가 WAS에서 시작해야 하는 이유

Day 2에서는 Flask의 `url_for()`를 통해 링크와 폼의 URL을 생성한다.

```
<a href="{{ url_for('detail_page', post_id=post.id) }}">
```

예를 들어 `url_for()`가 `/posts/6`을 반환하면 브라우저는 이 주소를 현재 접속 중인 호스트를 기준으로 해석한다.

브라우저 위치	실제 요청	결과
`localhost:5001`	`localhost:5001/posts/6`	Flask가 상세 페이지 응답
`localhost:8080`	`localhost:8080/posts/6`	Nginx에 해당 경로가 없어 404

글쓰기 폼도 동일하다.

```
<form method="post" action="{{ url_for('create_post') }}">
```

`url_for()`가 `/posts/new`을 반환하면 브라우저는 현재 호스트에 POST 요청을 보낸다. 따라서 브라우저가 `8080`에 있다면 해당 요청은 정적 파일 서버인 Nginx로 전달된다. Nginx는 Flask의 라우팅이나 비즈니스 로직을 처리하지 않기 때문에 404 또는 405 오류가 발생할 수 있다.

따라서 Day 2에서는 브라우저가 WAS인 `5001`에서 시작해야 이후의 링크와 폼 요청이 Flask의 라우팅과 정상적으로 연결된다.

이 때문에 `web` 컨테이너의 Welcome 페이지는 실제 애플리케이션 화면을 제공하기보다는 Flask 애플리케이션으로 이동하기 위한 최초 진입점의 역할을 한다.

3.2.3 컨테이너별 역할 변화

렌더링 주체가 변경되면서 세 컨테이너의 역할도 다음과 같이 변화하였다.

컨테이너	Day 1 - 방법 1	Day 2 - 방법 3
web	HTML + JavaScript 제공 및 화면 구성	Welcome 페이지 제공
was	JSON API 제공	HTML 생성 + 비즈니스 로직 + DB 연동
db	데이터 저장	데이터 저장

특히 Day 2에서 `templates/` 디렉터리가 `was` 내부에 위치하게 된 것이 이러한 변화를 잘 보여준다. Flask가 데이터 조회뿐만 아니라 HTML 생성까지 담당하면서 애플리케이션의 주요 책임이 WAS에 집중되었다.

3.2.4 구조 변경으로 얻은 점

첫 번째 장점은 CORS 설정이 필요 없어졌다는 것이다.

Day 1에서는 `8080`에서 실행되는 JavaScript가 `5001`의 API를 호출했기 때문에 서로 다른 출처 간 통신을 허용하기 위한 CORS 설정이 필요했다.

```
Day 1
8080 (Web) ──── API 요청 ────→ 5001 (WAS)
          서로 다른 출처

Day 2
5001 (WAS) ── 페이지/요청 ──→ 5001 (WAS)
          동일한 출처
```

Day 2에서는 Flask가 `5001`에서 페이지를 생성하고 이후의 링크와 요청도 동일한 출처에서 처리하므로 기존의 CORS 설정을 제거할 수 있었다.

두 번째 장점은 초기 화면 로딩 과정이 단순해졌다는 것이다.

Day 1은 `HTML 수신 → JavaScript 실행 → API 요청 → JSON 응답 → DOM 구성`의 여러 단계를 거쳤지만, Day 2에서는 Flask가 서버에서 데이터를 조회하고 HTML까지 생성하기 때문에 한 번의 페이지 요청으로 완성된 화면을 전달할 수 있다.

3.2.5 구조 변경으로 잃은 점

반면 서버에서 전체 HTML을 생성하는 방식은 페이지 전체가 다시 렌더링되는 단점이 있다.

예를 들어 검색이나 글 작성과 같은 요청이 발생하면 변경된 일부 영역만 갱신하는 것이 아니라 새로운 HTML 전체를 전달받는다. 따라서 Day 1에서 JavaScript를 통해 구현할 수 있었던 부분적인 화면 갱신이나 부드러운 사용자 경험은 상대적으로 떨어진다.

또한 web 컨테이너의 역할이 크게 축소되었다. 실제 애플리케이션 화면을 Flask가 생성하기 때문에 현재 구조에서는 Nginx가 Welcome 페이지 한 장을 제공하는 역할에 가까워졌다.

실제 운영 환경에서는 Nginx를 단순한 정적 파일 서버가 아니라 Reverse Proxy로 구성하여 외부 요청을 WAS로 전달하거나 TLS 종료, 정적 파일 처리 등의 역할을 담당하도록 구성할 수 있다. 따라서 현재 Day 2의 구조는 학습 목적으로 각 컨테이너의 역할과 서버 사이드 렌더링 구조를 확인하기에 적합하지만, 실무에서는 Nginx의 역할을 보다 확장하는 방향으로 발전시킬 수 있다.

3.4. Why Method 3

3.4.1. 백엔드의 역할을 명확하게 확인하기 위해

방법 1·2에서는 WAS가 JSON을 반환하고, 실제 화면 구성은 FE(JavaScript)가 담당한다.

```
방법 1·2
Browser → WAS → JSON
Browser → JSON을 이용해 화면 생성
```

반면 방법 3에서는 Flask가 DB에서 데이터를 가져와 HTML까지 완성해서 전달한다.

```
방법 3
Browser → WAS → DB
            ↓
          HTML 생성
            ↓
           Browser
```

따라서 이번 실습에서 Flask가 라우팅 → 비즈니스 로직 → DB 조회 → HTML 렌더링까지 처리하는 백엔드의 전체 흐름을 확인하기 좋다.

3.4.2. Docker 컨테이너 간 역할을 이해하기 좋음

이번 실습의 핵심이 `web → was → db` 구조를 이해하는 것이라면 방법 3이 직관적이다.

- web: Welcome 페이지 제공
- was: 실제 웹 서비스 및 비즈니스 로직 담당
- db: 데이터 저장

특히 `templates/`가 WAS 안에 존재하기 때문에 **"화면을*생성하는 책임이 WAS로 넘어갔다"**는 것을 직접 확인할 수 있다.

3.4.3. CORS와 FE 라우팅을 별도로 신경 쓰지 않아도 됨

방법 1·2에서는 FE와 BE가 서로 다른 출처에서 통신하기 때문에 CORS, API 통신, FE 라우팅 등을 추가로 고려해야 한다.

반면 방법 3에서는 브라우저가 WAS에 접속하고 Flask가 페이지와 라우팅을 모두 처리한다.

```
브라우저
   ↓
Flask
 ├ /posts
 ├ /posts/1
 ├ /posts/new
 └ DB
```

그래서 백엔드와 Docker의 기본 구조를 학습하는 단계에서는 훨씬 단순하다.